

RTL with Verilog (Week#3)

PS/2 Port:

A **PS/2 (Personal System/2) port** is a **6-pin mini-DIN connector** developed by IBM for connecting **keyboards and mice** to a computer. It is a serial communication interface that transfers data one bit at a time.

Purpose of a PS/2 Port

The PS/2 port is mainly used to connect:

- Keyboard
- Mouse

Before USB became common, PS/2 was the standard interface for these devices.

How Does a PS/2 Port Work?

Communication uses **two wires**:

- **Clock line (ps2_clk)**
- **Data line (ps2_data)**

Unlike UART, the keyboard itself generates the clock.

Design:

```
module ps2_receiver(  
    input ps2_clk,  
    input rst,  
    input ps2_data,  
  
    output reg [7:0] scan_code,  
    output reg data_valid,  
    output reg parity_error,  
    output reg frame_error  
);
```

```
reg [3:0] bit_count;
```

```
reg [7:0] data_reg;
```

```
reg parity;
```

```
reg [1:0] state;
```

```
localparam IDLE  = 2'd0;
```

```
localparam DATA = 2'd1;
```

```
localparam PARITY = 2'd2;
```

```
localparam STOP  = 2'd3;
```

```
always @(negedge ps2_clk or posedge rst)
```

```
begin
```

```
    if(rst)
```

```
    begin
```

```
        state <= IDLE;
```

```
        bit_count <= 0;
```

```
        data_reg <= 0;
```

```
        scan_code <= 0;
```

```
        data_valid <= 0;
```

```
        parity_error <= 0;
```

```
        frame_error <= 0;
```

```
    end
```

```
else
```

```

begin

    data_valid <= 0;

    case(state)

        //-----

        // Wait for Start Bit

        //-----

        IDLE:

            begin

                if(ps2_data == 0)

                    begin

                        bit_count <= 0;

                        parity_error <= 0;

                        frame_error <= 0;

                        state <= DATA;

                    end

                end

            end

        //-----

        // Receive 8 Data Bits (LSB First)

        //-----

        DATA:

            begin

                data_reg[bit_count] <= ps2_data;

```

```
    if(bit_count == 7)

        state <= PARITY;

    else

        bit_count <= bit_count + 1;

    end
```

```
//-----
```

```
// Receive Parity Bit
```

```
//-----
```

```
PARITY:
```

```
begin
```

```
    parity <= ps2_data;
```

```
    if((^data_reg) == ps2_data)
```

```
        parity_error <= 1;
```

```
    state <= STOP;
```

```
end
```

```
//-----
```

```
// Receive Stop Bit
```

```
//-----
```

```
STOP:
```

```
begin
```

```

        if(ps2_data != 1'b1)

            frame_error <= 1;

            scan_code <= data_reg;

            data_valid <= 1;

            state <= IDLE;

        end

    endcase

end

endmodule

```

TestBench:

```

`timescale 1us/1us

module ps2_receiver_tb;

    // Inputs

    reg ps2_clk;

    reg rst;

    reg ps2_data;

    // Outputs

    wire [7:0] scan_code;

    wire data_valid;

```

```

wire parity_error;

wire frame_error;


// DUT

ps2_receiver dut(

    .ps2_clk(ps2_clk),

    .rst(rst),

    .ps2_data(ps2_data),

    .scan_code(scan_code),

    .data_valid(data_valid),

    .parity_error(parity_error),

    .frame_error(frame_error)

);


//-----

// Clock Generation (100 us period)

//-----

initial begin

    ps2_clk = 1'b1;

    forever #50 ps2_clk = ~ps2_clk;

end


//-----

// Send one PS/2 frame

//-----

```

```

task send_ps2_byte;

    input [7:0] data;

    integer i;

    reg parity;

begin

    // Odd parity

    parity = ~(^data);

    //-----

    // Start Bit

    //-----

    ps2_data = 0;

    @(negedge ps2_clk);

    @(posedge ps2_clk);

    //-----

    // Data Bits (LSB First)

    //-----

    for(i=0;i<8;i=i+1)

    begin

        ps2_data = data[i];

        @(negedge ps2_clk);

        @(posedge ps2_clk);

```

end

//-----

// Parity Bit

//-----

ps2_data = parity;

@(negedge ps2_clk);

@(posedge ps2_clk);

//-----

// Stop Bit

//-----

ps2_data = 1'b1;

@(negedge ps2_clk);

@(posedge ps2_clk);

// Idle

ps2_data = 1'b1;

end

endtask

//-----

// Test Sequence

//-----

initial begin

rst = 1'b1;

ps2_data = 1'b1;

#200;

rst = 1'b0;

#200;

\$display("Sending Scan Code 0x1C");

send_ps2_byte(8'h1C);

wait(data_valid);

#100;

\$display("-----");

\$display("Scan Code = %h",scan_code);

\$display("Data Valid = %b",data_valid);

\$display("Parity Error = %b",parity_error);

\$display("Frame Error = %b",frame_error);

```

        if(scan_code==8'h1C &&

            parity_error==0 &&

            frame_error==0)

            $display("TEST PASSED");

        else

            $display("TEST FAILED");

        #500;

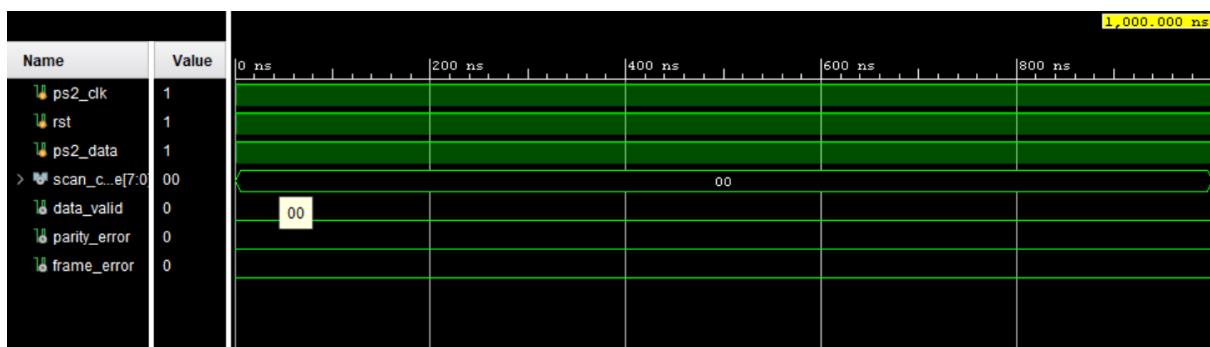
        $finish;

    end

endmodule

```

Wave Form :



Synthesis :

